FinSight

Technical project guide

Architecture, workflow, agent design, safety controls, and limitations

The root README contains the beginner setup instructions. This PDF explains how the finished system works.

FinSight technical project guide

Purpose

FinSight is a local financial-document question-answering assistant. It reads supported financial files, converts their content into searchable text, retrieves relevant evidence, and asks Gemini to explain that evidence through one LangChain agent.

The beginner setup and running instructions are in the root `README.md`. This guide focuses on how the system is designed and what its boundaries are.

Architecture

```
Streamlit interface
        |
Upload validation
        |
PDF/TXT/CSV/Excel readers
        |
Text chunks and table labels
        |
Gemini embeddings -> Chroma vector search
        |
Relevant evidence
        |
One LangChain agent and one search tool
        |
Grounded answer with a source note
```

When Gemini embeddings are unavailable, the project keeps a simple local keyword-search fallback so document reading and basic retrieval can still be demonstrated.

Main components

Component	File or library	Responsibility
Browser interface	`app.py`, Streamlit	Uploads, mapping review, chat, status messages, and workspace reset
Configuration	`src/config.py`, python-dotenv	Paths, model names, limits, and Gemini key
Document reader	`src/documents.py`, pypdf, pandas	Validates files and extracts text or tables
Label mapping	`src/mapping.py`	Recognizes common labels such as Revenue and Net profit
RAG index	`src/rag.py`, LangChain, Chroma	Chunks text, creates embeddings, stores chunks, and retrieves evidence
Agent	`src/agent.py`, LangChain, Gemini	Applies scope rules and writes an evidence-grounded response
Workspace storage	`src/storage.py`	Saves uploads, metadata, and vector data and clears them together
Error types	`src/errors.py`	Gives the UI clear unsupported and invalid-file errors

Workflow

1. Streamlit starts and creates the runtime folders.
2. The app restores accepted files from `data/uploads/` when the workspace already contains them.
3. A selected file is checked for extension, size, basic content, and readable text.
4. The reader returns one `DocumentRecord` with the file name, extracted text, tables, and warnings.
5. The RAG layer splits text into overlapping chunks. Overlap helps preserve facts that fall near a chunk boundary.
6. With a Gemini key, the chunks are embedded and stored in the local Chroma database. Without it, the local keyword fallback remains available.
7. A table label helper identifies common financial fields for the Analysis section. It does not calculate ratios or change the source document.
8. A question is checked for empty input, missing documents, unsupported scope, and investment advice.
9. Relevant chunks are retrieved. Overview questions use a small sample from every uploaded source.
10. The single LangChain agent receives the question and retrieved evidence. Its prompt tells it to use only that evidence, separate facts from possible insights, and state when evidence is insufficient.
11. The UI streams the answer and adds one concise file-level source note.

Local deployment

FinSight is intentionally delivered as a localhost Streamlit application. After the user creates the Python environment, installs `requirements.txt`, and adds the Gemini key to `.env`, the application is started with:

```
python -m streamlit run app.py
```

Streamlit serves the interface at `http://localhost:8501`. No cloud hosting or public deployment is required for this submission. The complete setup and run steps are in `README.md`.

Agent and tool design

The project uses one agent because the workflow has one clear responsibility: answer questions from uploaded documents. The agent has one tool named `search_documents`. That tool searches the same `SearchIndex` used by the application and returns text with source names.

The rest of the functions are ordinary application code. File validation, parsing, chunking, mapping, storage, and UI rendering do not need to be agents or tools.

Reliability and safety controls

- Extension, size, empty-content, and basic PDF checks happen before indexing.
- Unsupported or unreadable files are shown in the rejected list and are not added to the index.

- The prompt places retrieved content between evidence markers and tells Gemini not to invent figures, causes, predictions, or recommendations.
- The answer separates facts stated in the files from a possible insight when the user asks for a cause or management concern.
- Missing evidence produces a clear response instead of a guessed answer.
- Investment, price-prediction, legal, tax, and audit requests stay outside the assistant's scope.
- Gemini and Chroma failures are caught and returned with available evidence where possible.
- The Clear workspace action removes disk data, mappings, chat history, and the in-memory index.

Limitations

- Image-only PDFs have no OCR in this version. Upload a searchable PDF or export the data to CSV/Excel.
- The assistant has no live exchange-rate, weather, market-price, tax, legal, or audit data.
- A private abbreviation or unusual table label may need user mapping correction.
- Conflicting values across files are not automatically reconciled. Retrieval may return one or more conflicting chunks without identifying every disagreement. Use documents for one company and review conflicting figures manually.
- The current chat box has no explicit character limit. Very long questions can exceed the model provider's request limit or respond slowly. Short, focused questions are more reliable.
- Answers depend on the quality and completeness of the uploaded documents. The assistant cannot prove a cause that the files do not describe.
- Gemini chat and semantic embeddings need a valid key, internet access, an available model, and free-tier quota. The local reader and keyword fallback can still demonstrate the non-LLM path.

Challenges faced during development

- Supporting PDF, TXT, CSV, and Excel files required separate validation and extraction paths because each format stores information differently.
- Financial labels and table layouts vary between documents, so the Analysis section uses simple label matching and allows the user to correct an unclear mapping.
- Gemini and Chroma depend on a valid key, internet access, an available model, and quota. A local keyword-search fallback was kept so document retrieval can still be demonstrated when semantic indexing is unavailable.
- The application must avoid inventing a cause when the uploaded files only show figures. The prompt and response format therefore separate reported facts, possible insights, and missing evidence.
- Documents from different companies can contain conflicting values, and the current version reports retrieved evidence while asking the user to review conflicts manually.

Project structure

```
FinSight/
├── app.py
├── requirements.txt
├── .env
├── .env.example
├── README.md
├── src/
├── tests/
├── samples/
├── docs/
└── data/                  created at runtime
    ├── uploads/
    ├── processed/
    └── vector_db/
```

The runtime folders are created automatically and should not contain generated files in the final submission ZIP. The separate `manual_test_files/` folder is for local boundary testing and should also be excluded from the LMS ZIP.

Validation reference

Use `python -m pytest -q` for the automated checks. The current test suite covers supported and unsupported readers, empty and invalid files, mapping, chunking, fallback retrieval, no-document handling, out-of-scope handling, overview retrieval, and source references. The visible manual questions and expected behavior are in `docs/TEST_CASES.md`; the recorded run is in `docs/TEST_REPORT.md`.